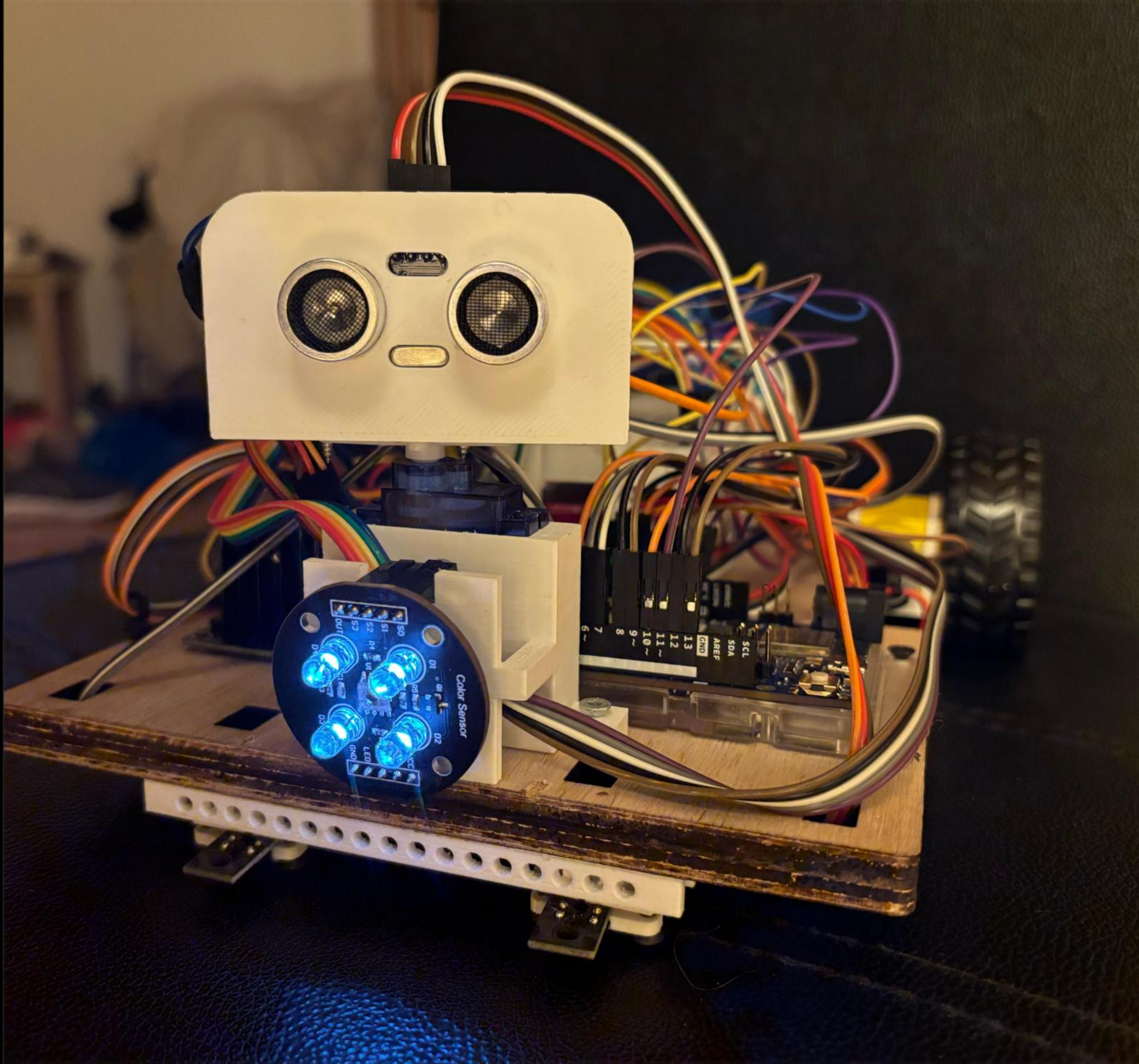
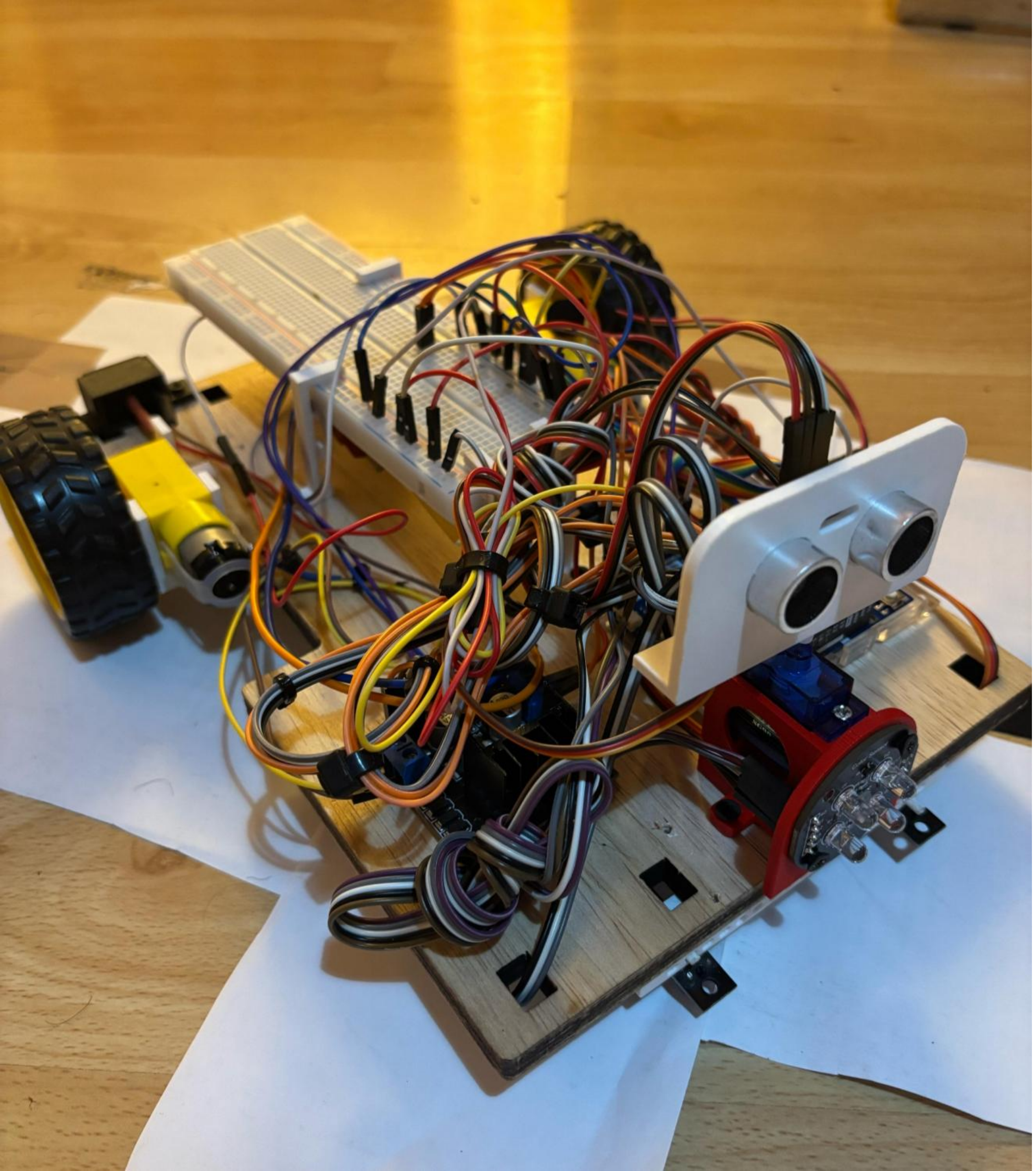
SYSTEM ENGINEERING & PROTOTYPING

Team: C1

Members:
- Md Mostafizur Rahman
- Mohamed Abdo
- Md Ruhul Kuddus Saleh
- M Amir Ejaz

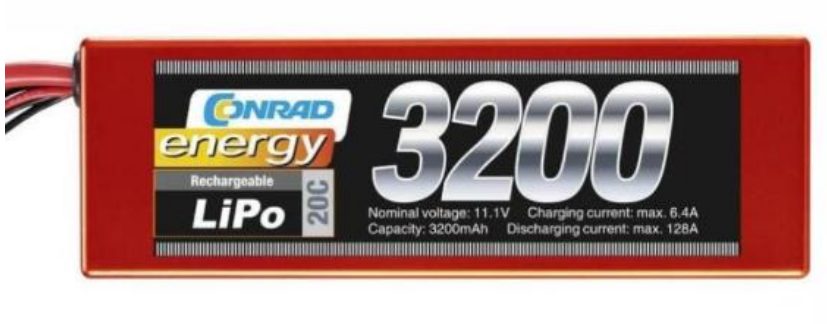




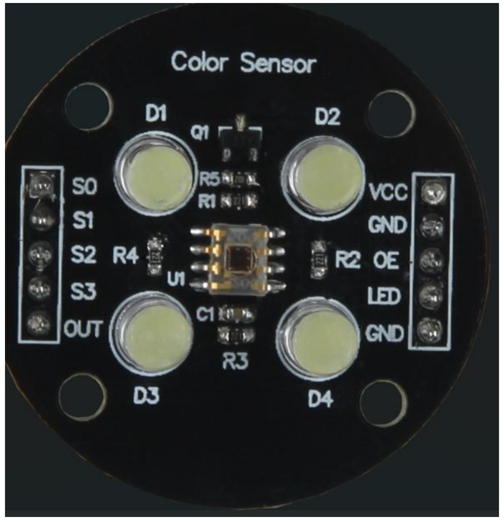
SYSTEM OVERVIEW

- Developing an autonomous car using Arduino.
- Integrates ultrasonic sensors for obstacle detection
- Infrared sensors for line following
- Uses color sensor for for color detection
- DC motors drive the car.
- Servo motor for obstacle removal

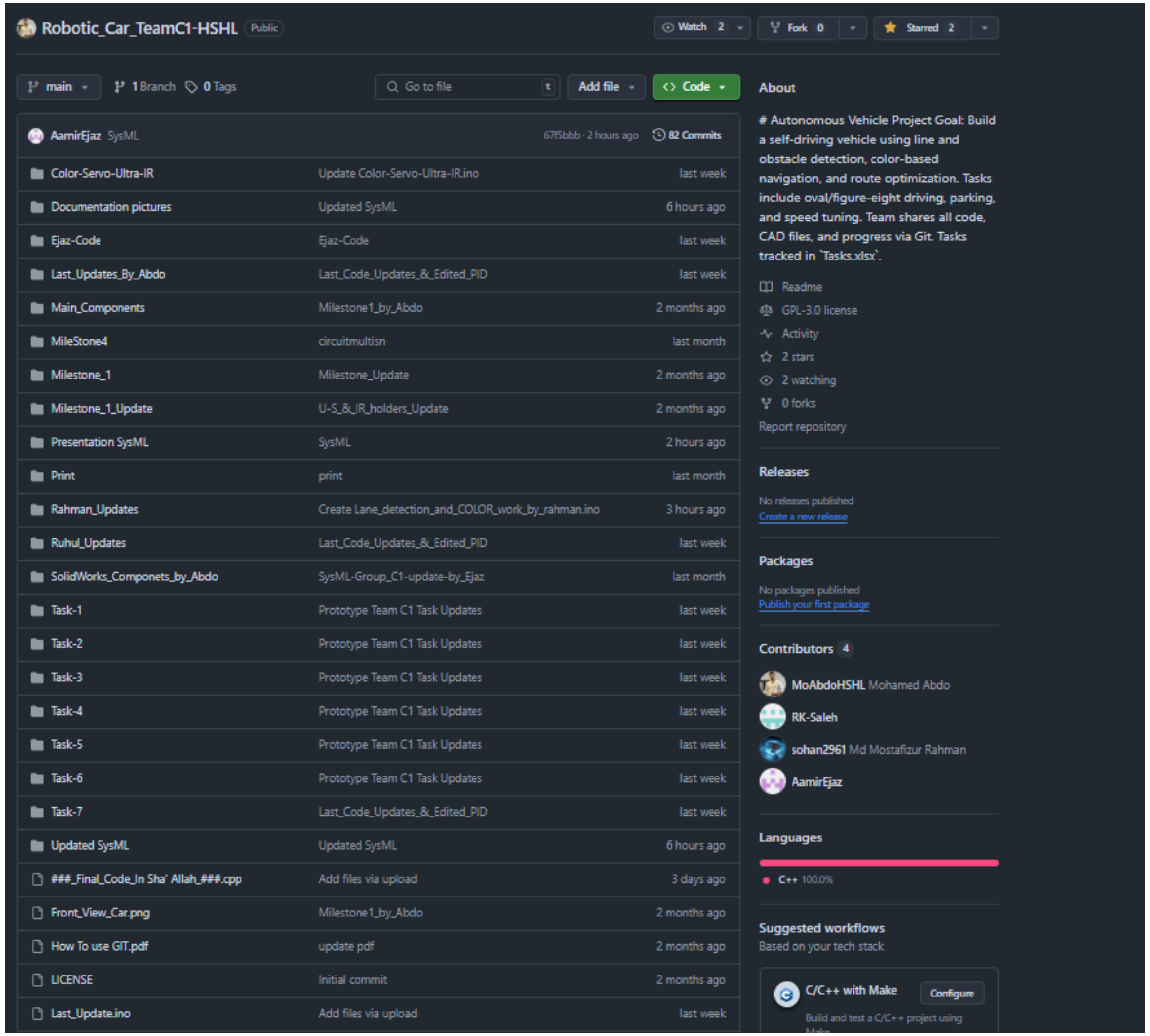
COMPONENTS


CONRAD
energy
Rechargeable
LiPo
20C
3200
Nominal voltage: 11.1V Charging current: max. 6.4A
Capacity: 3200mAh Discharging current: max. 128A


SR04
T
R
Vcc
Trig
Echo
Gnd


Color Sensor
D1
D2
Q1
R5
R1
S0
S1
S2
S3
OUT
R4
U1
C1
R3
R2
VCC
GND
OE
LED
GND
D3
D4

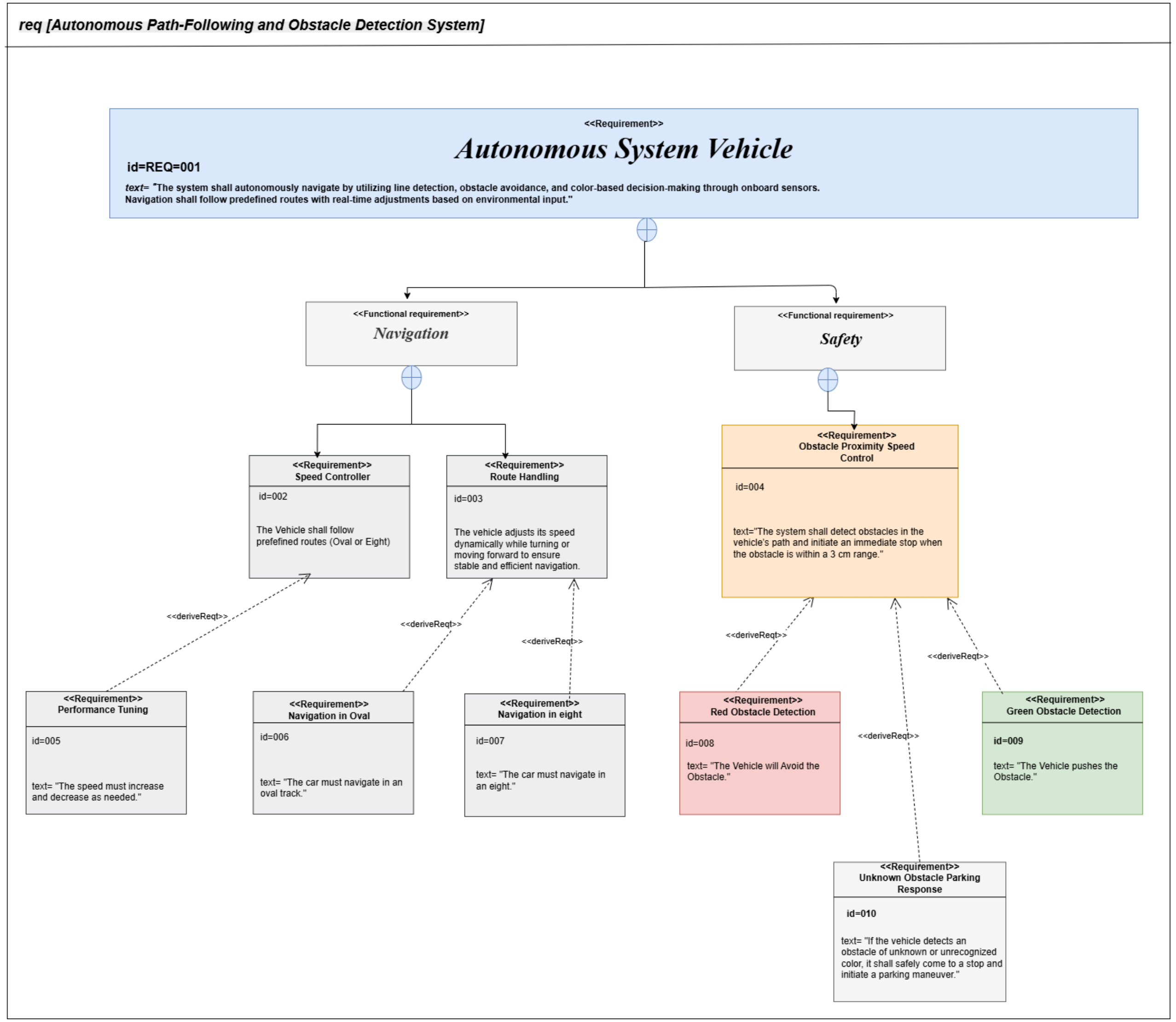

OUT2
ENA IN1 IN2 IN3 IN4 ENB



TEAM CONTRIBUTION & GIT COMMITS

https://github.com/MoAbdoHSHL/Robotic_Car_TeamC1-HSHL

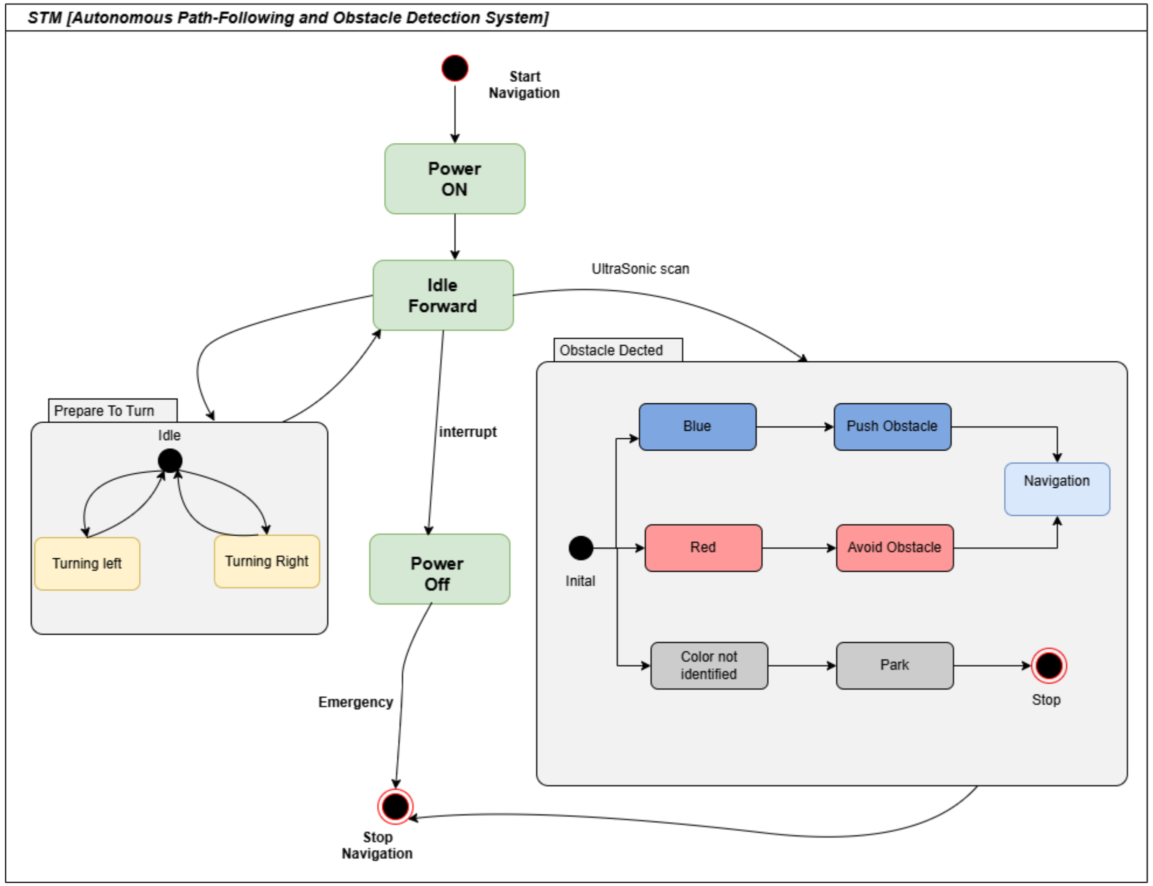
Requirement

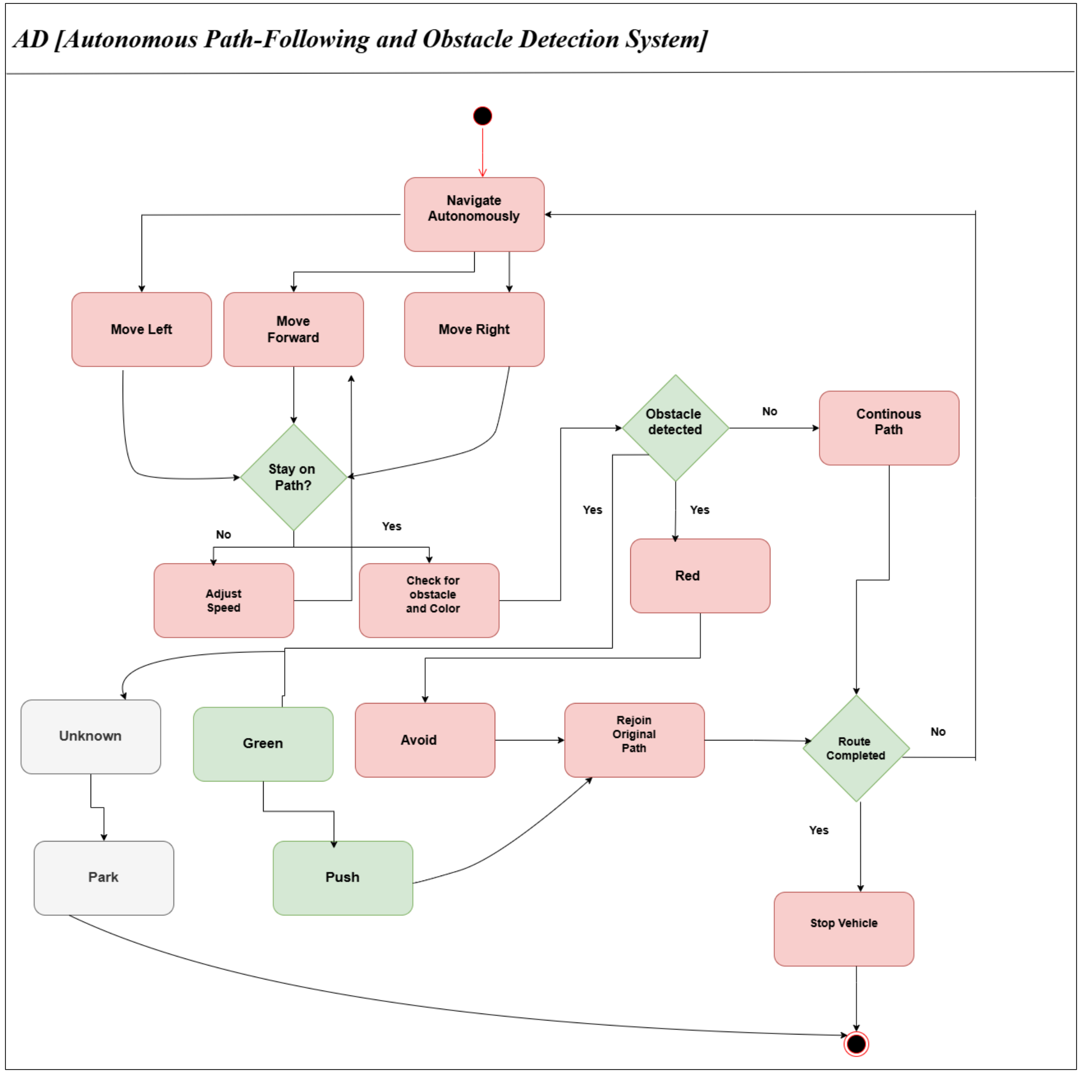


- Follow a lane
- Drive different routes
- Optimize Voltage and Speed
- Detect Obstacles
- Detect Different color and take actions
- Push / Overtake an obstacle
- Park the car

STATE MACHINE DIAGRAM

- Starts with "Power ON,,
- Moves to "Idle Forward,,
- Detects obstacles and color.
- Responds (push, avoid, or park).
- Handles turning, interrupts, and emergencies.

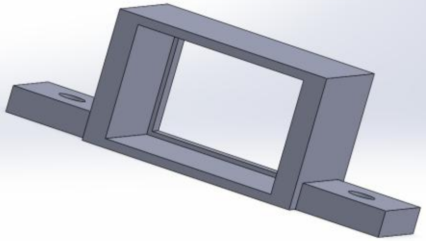




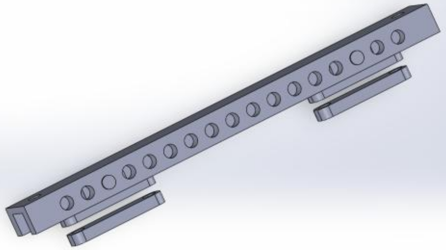
ACTIVITY DIAGRAM

- Starts with autonomous navigation.
- Decides to move left, right, or forward.
- Checks if it's on the path.
- Detects obstacles & responds (avoid, push, or stop).
- Ends by stopping or parking.

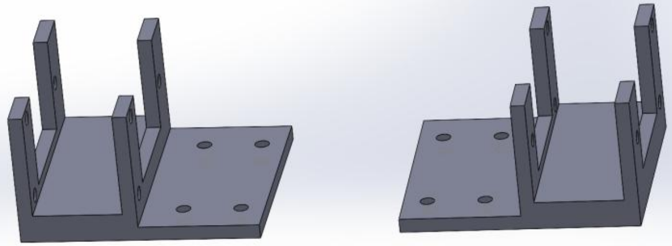
3D PRINTED PARTS



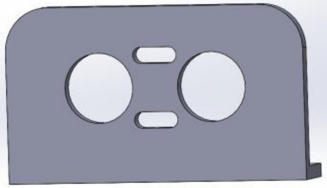
Switch Holder



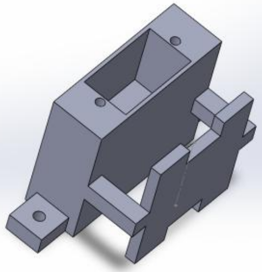
IR Holder



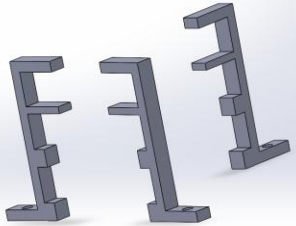
Motor Holder



Ultrasonic Holder

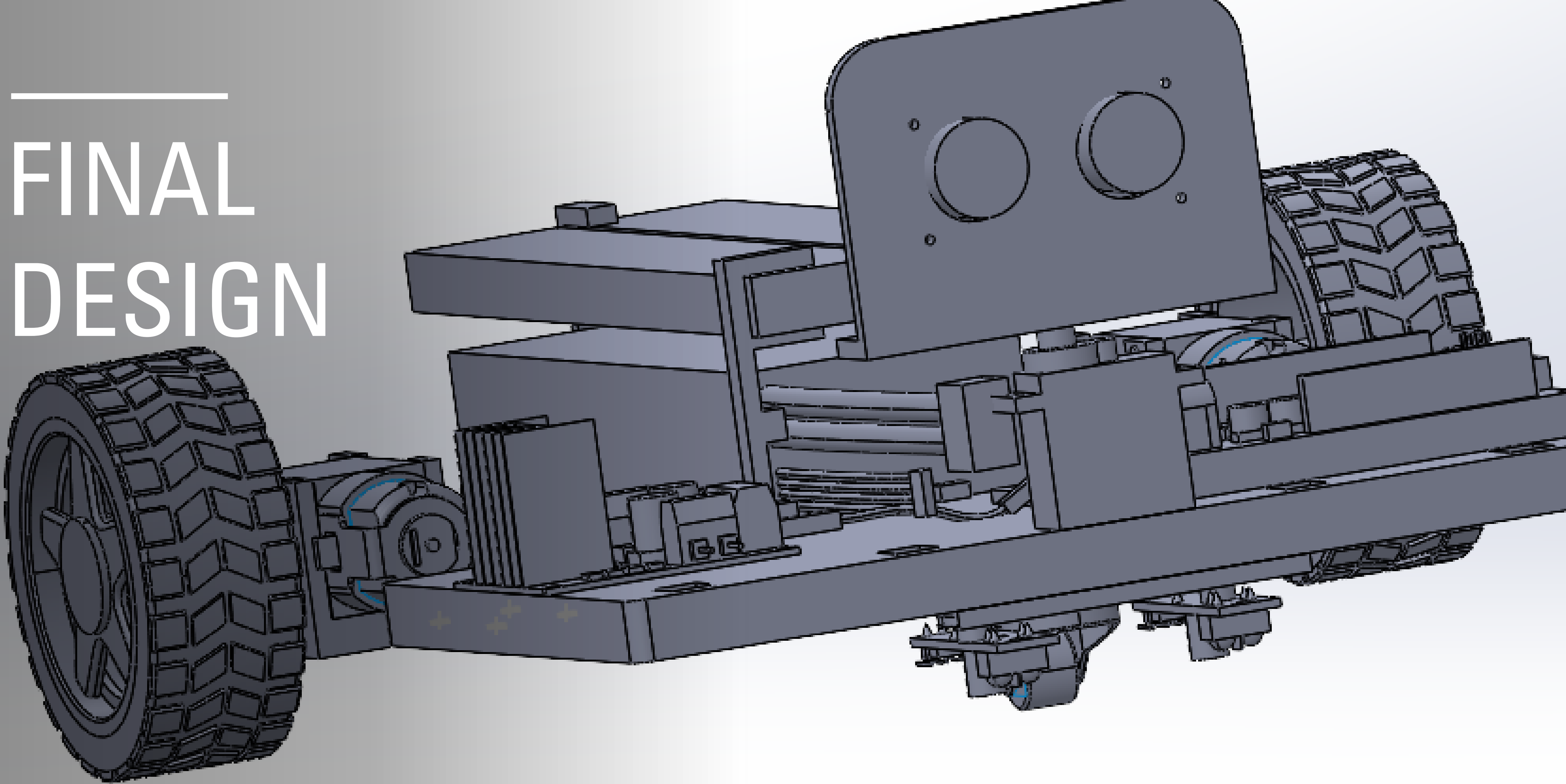


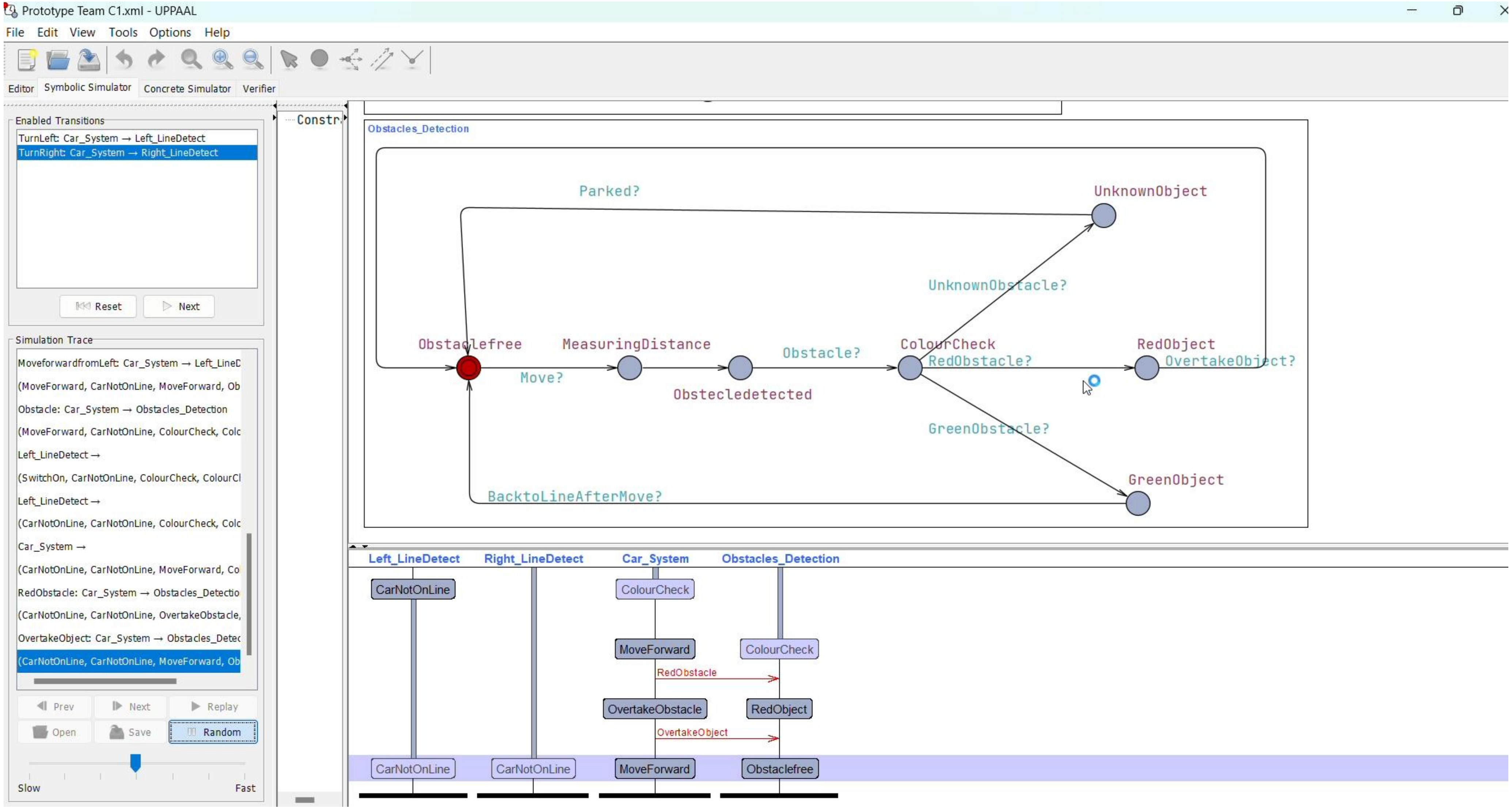
Color & Servo Holder



Battery Holder

FINAL DESIGN

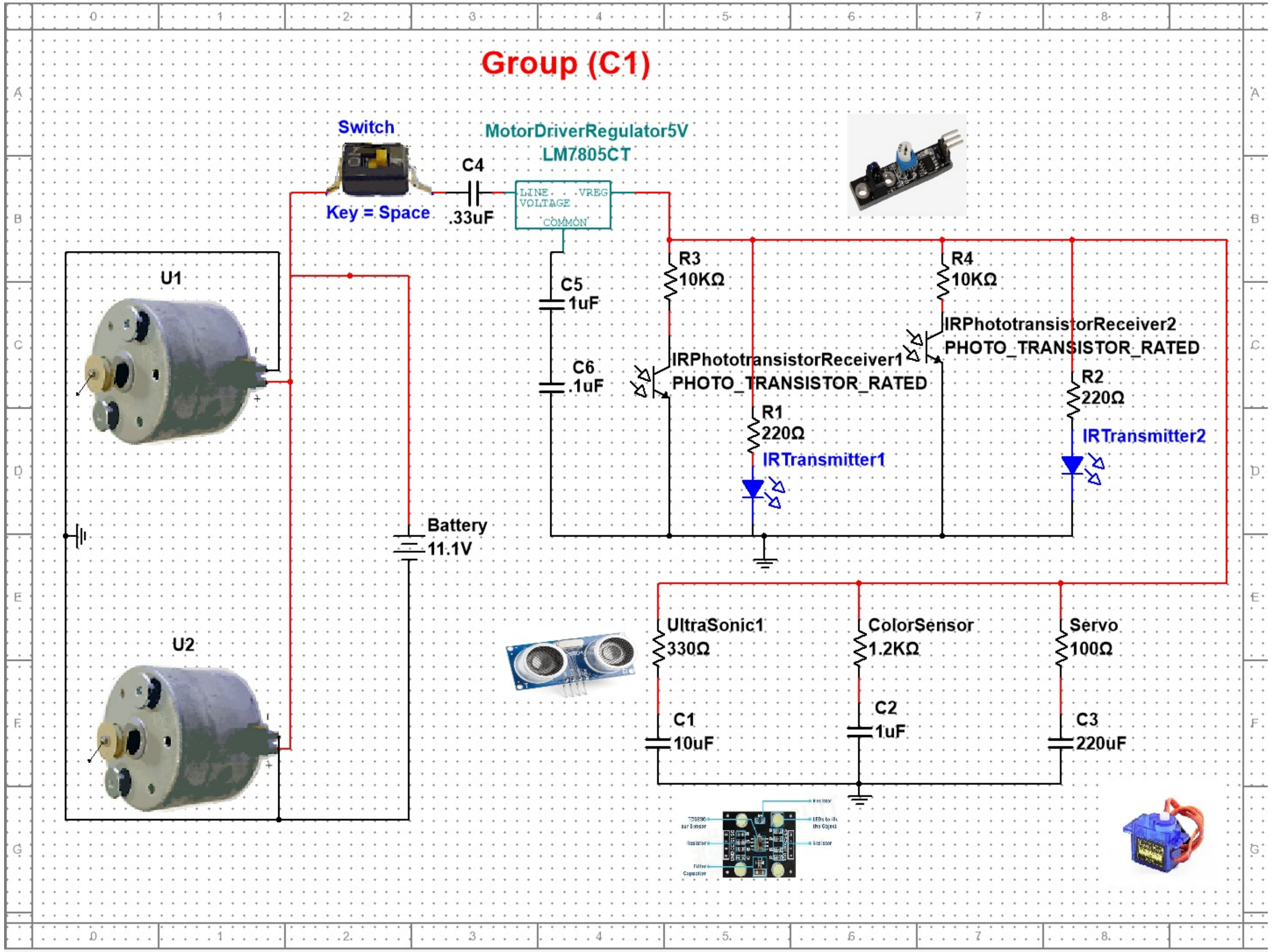




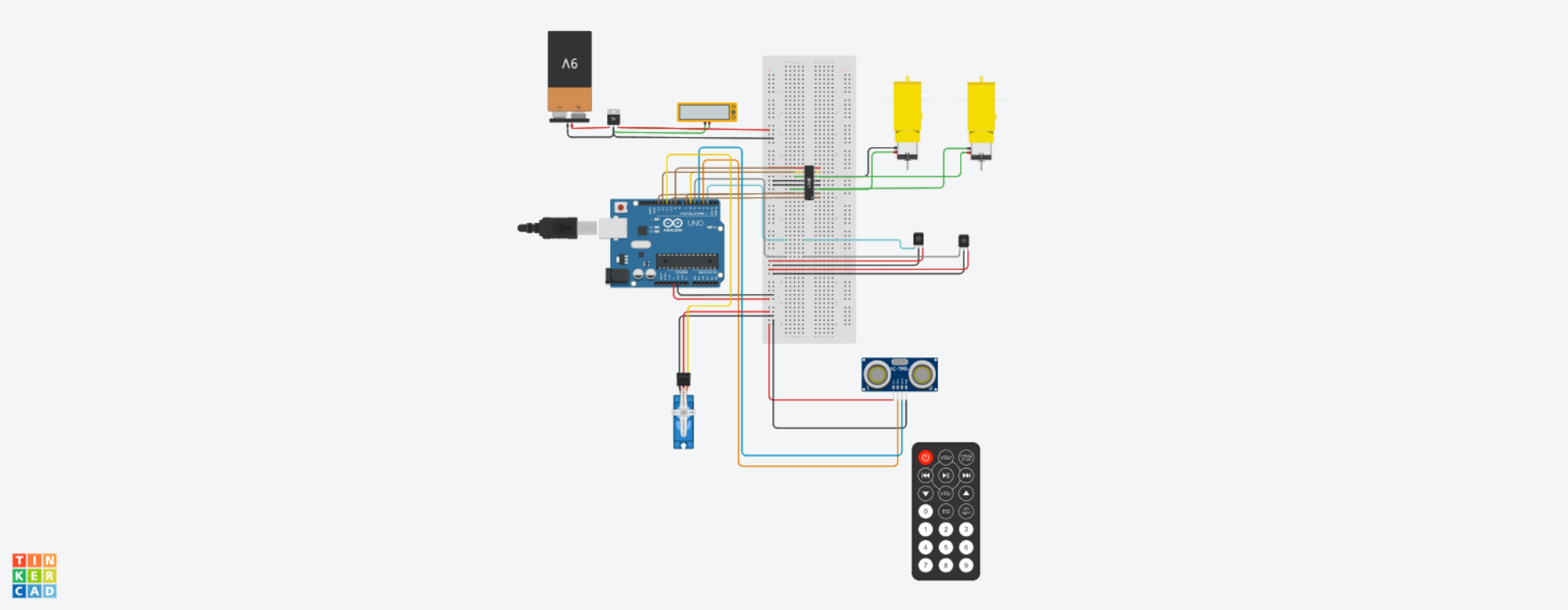
UPPAL SIMULATION

VOLTAGE & CURRENT MEASUREMENT

- Uses motors, sensors, and a battery.
- voltage regulator and capacitors.
- IR and color sensors detect obstacles.
- Power flows through regulated paths.



SCHEMATIC VIEW ON TINKERCAD



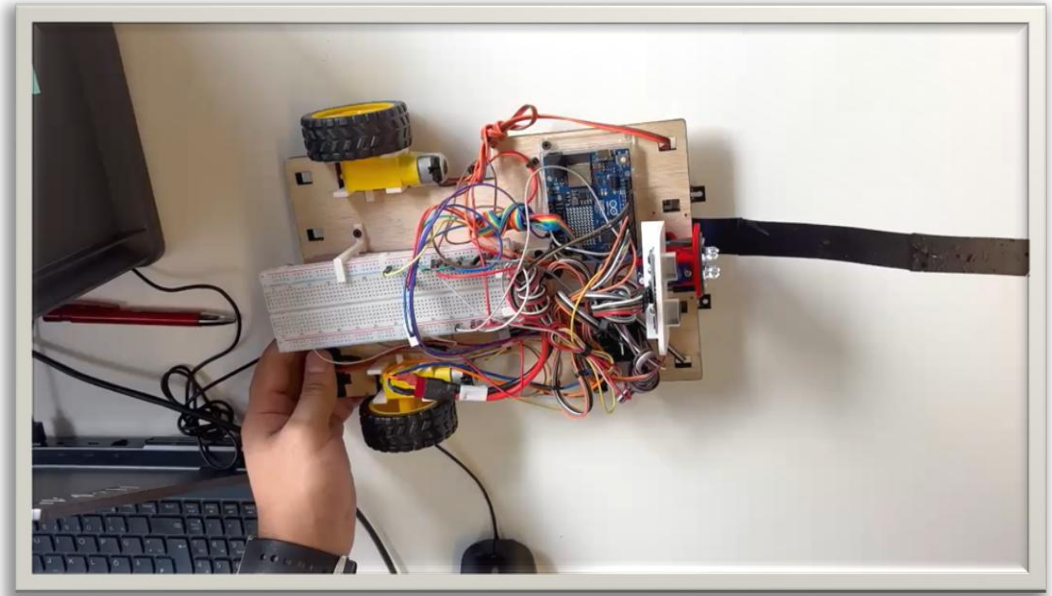
https://www.tinkercad.com/things/dzwxTrQqEL3-swanky-snaget/editel?returnTo=https%3A%2F%2Fwww.tinkercad.com%2Fdashboard&sharecode=xYkdW4VoKbrRX9IXlcNGk5c59ltT1bGMOaSqg_Oswtw

- wiring connectins.
- Arduino controls

  sensors and motors
- Initial Code testing

LANE FOLLOWING

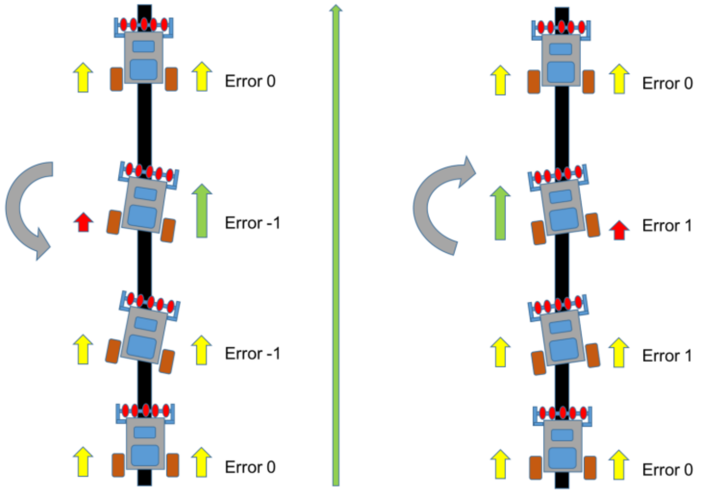
```
  if (leftIR == 0 && rightIR == 0) {
    moveForward(speedA, speedB);
    analogWrite(enA, forwardSpeed);
    analogWrite(enB, forwardSpeed);
    forward();
  } else if (leftIR == 1 && rightIR == 0) {
    leftBackward(60);
    rightForward(80);
    curveLeft();
    Stop(); delay(30);

  } else if (leftIR == 0 && rightIR == 1) {
    leftForward(80);
    rightBackward(60);
    curveRight();
    Stop(); delay(30);
  } else {
    Stop();
    delay(100);
  }
}
```



```
void loop() {
  int leftIR = digitalRead(L_S);
  int rightIR = digitalRead(R_S);
```

PID FOR OPTIMIZING SYSTEM

```
float calculatePID(float currentVoltage) {
  unsigned long now = millis();
  float dt = (now - lastPIDTime) / 1000.0;
  if (dt <= 0) dt = 0.01;

  float error = batterySetpoint - currentVoltage;
  integral += error * dt;
  float derivative = (error - previousError) / dt;

  float output = kp * error + ki * integral + kd * derivative;

  previousError = error;
  lastPIDTime = now;

  return output;
}

int adjustPWM(int basePWM, float pidCorrection) {
  int adjustedPWM = basePWM + (int)pidCorrection;
  if (adjustedPWM > 255) adjustedPWM = 255;
  if (adjustedPWM < 0) adjustedPWM = 0;
  return adjustedPWM;
}
```



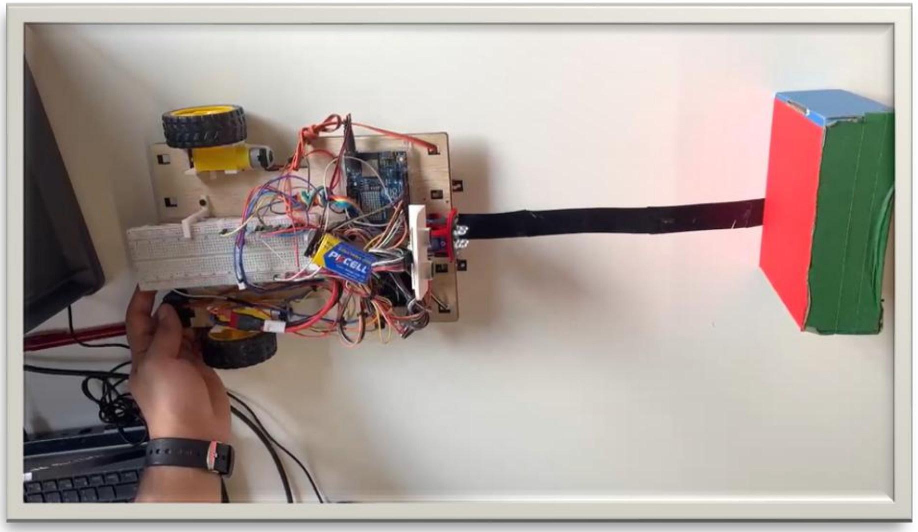

```
// PID parameters
float batterySetpoint = 7.4;  // Nominal battery voltage (adjust for your battery)
float kp = 20.0;
float ki = 0.1;
float kd = 5.0;

float integral = 0;
float previousError = 0;
unsigned long lastPIDTime = 0;

float pidOutput = 0;  // PID correction output
```

OBSTACLE DETECTION

```
bool ultrasonicEnabled = true;
```

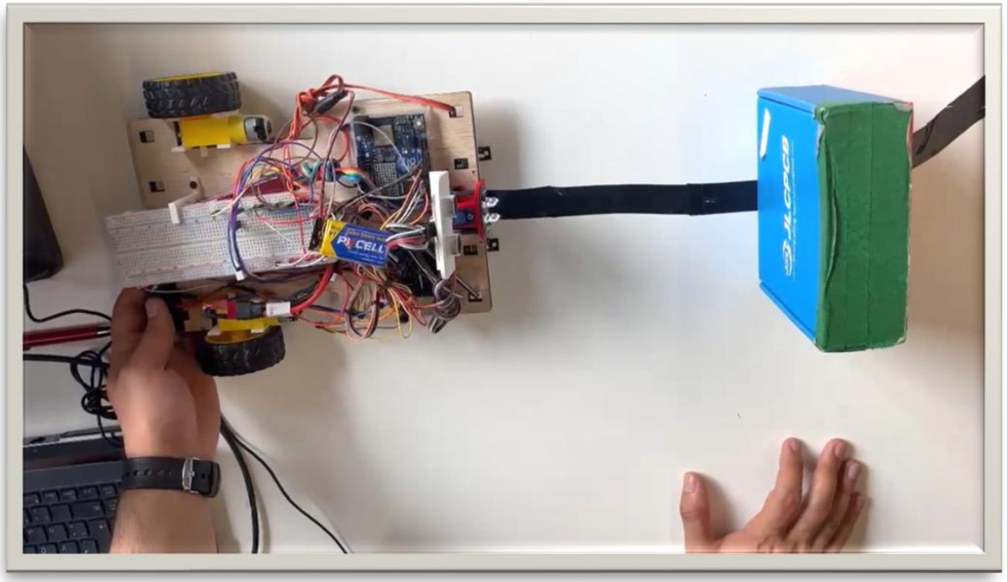
```
#define servoPin 4
#define SERVO_CENTER 70

Servo scanServo;
```

```
int getDistance() {
  if (!ultrasonicEnabled) return 1000;
  digitalWrite(trigPin, LOW); delayMicroseconds(2);
  digitalWrite(trigPin, HIGH); delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duration = pulseIn(echoPin, HIGH, 20000);
  return duration * 0.034 / 2;
}
```



COLOR DETECTION & REACTION

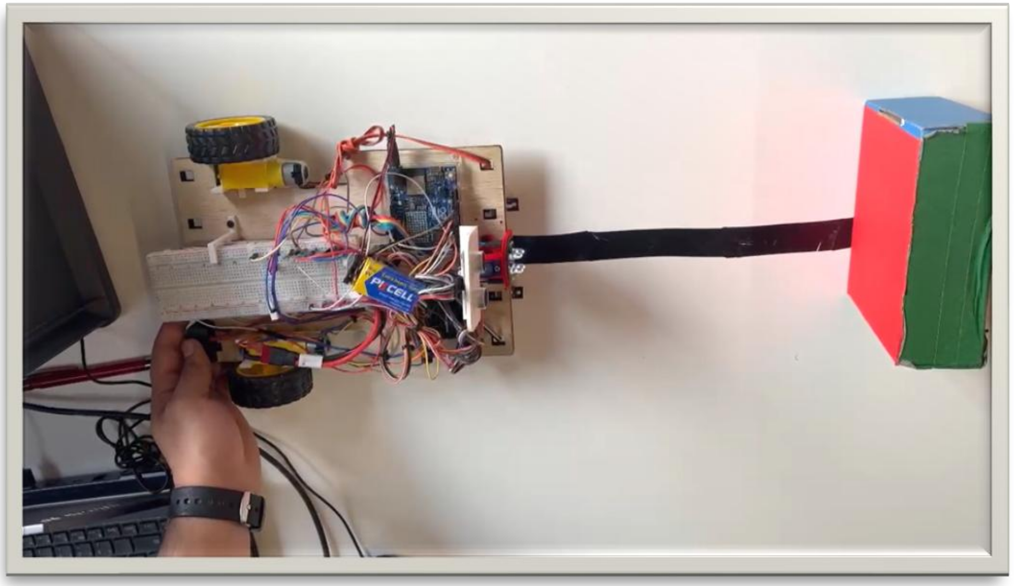
```
else if (color == "BLUE") {
  driveForward(20, 150);
  Stop(); delay(300);
  driveBackwards(30, 70);
  Stop(); delay(300);
  ultrasonicEnabled = false;
  unsigned long ignoreStart = millis();
  Serial.println("Ultrasonic disabled for 5 seconds...");
  while (millis() - ignoreStart < 5000) {
    int l = digitalRead(L_S);
    int r = digitalRead(R_S);
    if (l == 0 && r == 0) {
      analogWrite(enA, forwardSpeed);
      analogWrite(enB, forwardSpeed);
      forward();
    } else if (l == 1 && r == 0) {
      Stop(); delay(30);
      curveLeft();
    } else if (l == 0 && r == 1) {
      curveRight();
      Stop(); delay(30);
    } else {
      Stop();
      delay(100);
    }
    delay(20);
  }
  ultrasonicEnabled = true;
```



Blue Color Detected and Push the Object

COLOR DETECTION & REACTION

```
void handleDetectedColor(String color) {
  int colorScenarioSpeed = 80;
  if (color == "RED") {
    driveBackwards(12, colorScenarioSpeed);
    scanServo.write(180); delay(400);
    int leftDist = getDistance();
    scanServo.write(SERVO_CENTER); delay(200);
    if (leftDist > 20) {
      ultrasonicEnabled = false;
      rotate90(); delay(50);
      driveForward(30, colorScenarioSpeed); delay(200);
      rotateRight90(); delay(200);
      driveForward(20, colorScenarioSpeed);
      approachRightToLane();
      ultrasonicEnabled = true;
      Serial.println("RED: Path left is clear, performed left-right bypass.");
    } else {
      Serial.println("RED: Left not clear, only backed up and stopped.");
    }
    scanServo.write(SERVO_CENTER);
    Stop();
  }
```



Red Color Detected and turn 90° to avoid the Object & return back to lane

CHALLENGES & OBSERVATIONS

- Blue Detected
- Red Detected
- Green Not Detected

Table 1. Selectable Options

S0	S1	OUTPUT FREQUENCY SCALING (f_o)
L	L	Power down
L	H	2%
H	L	20%
H	H	100%

S2	S3	PHOTODIODE TYPE
L	L	Red
L	H	Blue
H	L	Clear (no filter)
H	H	Green

"TCS3200 Programmable Color Light-to-Frequency Converter," Texas Advanced Optoelectronic Solutions (TAOS), [Online]. Available: https://www.mouser.com/catalog/specsheets/tcs3200-e11.pdf. [Accessed: Jun. 27, 2024]


THANK YOU FOR YOUR TIME ☺
QUESTIONS OR FEEDBACKS?